UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

ESCOLA POLITÉCNICA DA UFRJ

MBA ENGENHARIA DE SOFTWARE

Ana Paula Zampier

Anny Caroline Sousa

Bruno Verdan

Daniel Szyfman

Paulo Victor Coelho

Victor Caldas

DevOnboard

Relatório Final

Rio de Janeiro, 22 de outubro de 2026

Disciplina: Ferramentas para Geração de Ideias (Ideação e Prototipagem de Soluções para a Indústria de Software)

Professora: Profa. Juliana França

Título do Projeto: DevOnboard: Solução Inovadora para Aceleração do Onboarding Técnico e Redução de Débito de Conhecimento na Indústria de Software

Integrantes: Ana Paula Zampier anapaulazampier@gmail.com; Anny Caroline Sousa sousa.annyc@gmail.com; Bruno Verdan brverdan@gmail.com; Daniel Szyfman szyfman023@gmail.com; Paulo Victor Coelho pvictorcoelho@gmail.com; Victor Caldas victorcaldas@gmail.com

Resumo

O DevOnboard é uma plataforma web que reduz o tempo de ramp-up de desenvolvedores recém-contratados. A solução combina um checklist interativo de setup, um mapa visual e navegável da arquitetura de sistemas e uma central de regras de negócio com dúvidas assíncronas. O trabalho segue o modelo do Diamante Duplo (Design Council), de Design Thinking, e o organiza em seis etapas: problema, empatia, ideação, seleção, prototipagem no Figma e métricas. A solução atende à etapa de manutenção e evolução do processo de engenharia de software, em especial à compreensão de arquitetura e à documentação. A meta projetada é reduzir o Time-to-First-PR em 60% e os chamados de suporte técnico interno em 45%.

Palavras-chave: onboarding técnico; experiência do desenvolvedor (DevEx); Design Thinking; arquitetura de software; prototipagem.

Nota metodológica: os dados quantitativos de mercado e os resultados de pesquisa e de validação com usuários apresentados neste relatório são simulados para fins acadêmicos. Foram construídos a partir de cenários plausíveis da indústria e servem para fundamentar a ideação e o protótipo. As metas da Etapa 6 são hipóteses a validar em um piloto.

Etapa 1 — Identificação do Problema e Relevância

Um desenvolvedor recém-contratado leva, em média, de 3 a 6 semanas para entregar seu primeiro commit relevante em produção. Nesse intervalo, a empresa paga salário integral por baixa produção e ainda consome horas dos engenheiros mais caros do time.

1.1 O problema: o ramp-up longo

Ramp-up é o período entre a contratação e o momento em que o profissional passa a entregar valor de forma autônoma. Na indústria de software, três fatores que se reforçam o alongam.

Documentação desatualizada ou fragmentada. O conhecimento técnico fica espalhado entre Confluence, READMEs, threads de Slack e a memória das pessoas. Como a documentação não acompanha o ritmo do código, ela envelhece rápido e perde a confiança do leitor. O novo desenvolvedor passa a perguntar em vez de ler.

Arquiteturas complexas e pouco visíveis. Sistemas monolíticos legados convivem com dezenas de microsserviços, filas, bancos e APIs internas. Raramente existe um diagrama atualizado de dependências. Entender "quem chama quem" exige ler código de vários repositórios ou interromper alguém.

Dependência do tempo dos sêniores. Sem uma fonte confiável, o conhecimento é transmitido oralmente. Tech Leads e engenheiros sêniores viram gargalo: respondem às mesmas perguntas a cada contratação, perdem foco e atrasam suas próprias entregas na sprint.

O resultado é um débito de conhecimento: assim como o débito técnico, ele acumula juros. Cada contratação paga o custo de redescobrir o que já foi descoberto antes.

1.2 Impacto na indústria

Os números abaixo são simulados e partem de um cenário de referência: empresa de tecnologia de porte médio, com 120 engenheiros e 30 contratações técnicas por ano.

Indicador	Valor de referência (simulado)
Tempo até o primeiro commit relevante em produção	3 a 6 semanas (média de 4,5)
Tempo até a produtividade plena	3 a 6 meses
Dias perdidos com setup do ambiente local	3 a 5 dias úteis
Horas de sênior consumidas por novo contratado no 1° mês	25 a 40 horas
Novos devs que relatam ansiedade ou fadiga no onboarding	68%
Perguntas repetidas ao time a cada contratação	cerca de 70% do total

Custo estimado de inatividade. Considerando um custo mensal total (salário e encargos) de R$ 20.000 por desenvolvedor pleno, 4,5 semanas de baixa produtividade a cerca de 30% de rendimento equivalem a aproximadamente R$ 14.500 de capacidade não entregue por contratação. Somando 30 horas de sênior a R$ 180/hora (R$ 5.400), o custo oculto chega perto de R$ 20.000 por novo desenvolvedor. Em 30 contratações por ano, são cerca de R$ 600.000 anuais.

Atrasos em sprints. O tempo de sênior desviado para dúvidas repetidas reduz a capacidade do time em 10% a 15% durante cada ciclo de contratação. Isso gera estimativas furadas, spillover de histórias e pressão sobre a previsibilidade das entregas.

Fadiga mental e retenção. Um onboarding confuso afeta a autoconfiança do profissional nas primeiras semanas, período em que se forma a percepção sobre a empresa. Experiências ruins de entrada estão associadas a maior turnover no primeiro ano, o que reinicia todo o custo de contratação.

1.3 Público-alvo

Perfil	Papel no onboarding	Dor principal
Desenvolvedor Frontend/Backend recém-contratado	Usuário primário	Não sabe por onde começar nem consegue rodar o projeto
Tech Lead	Guardião da arquitetura	Interrompido por perguntas repetidas; tempo de foco perdido
Engineering Manager	Responsável pelo resultado do time	Sem visibilidade do progresso; ramp-up imprevisível
Onboarding Mentor (buddy)	Acompanha o novo dev	Sem roteiro padronizado; depende da própria memória

Etapa 2 — Pesquisa e Empatia

A pesquisa indicou que o maior bloqueio do novo desenvolvedor não é técnico, e sim de orientação: ele não sabe onde está a informação confiável nem a quem perguntar sem constrangimento.

2.1 Metodologia de investigação

A fase de empatia combinou duas técnicas qualitativas ao longo de três semanas.

Investigação contextual. A técnica segue a Contextual Inquiry de Holtzblatt e Beyer (2017), no modelo mestre-aprendiz. Cada integrante consultou, nas empresas em que trabalha, supervisores e líderes técnicos sobre a rotina de desenvolvedores nos primeiros 30 dias em duas empresas de tecnologia. Os supervisores relataram tarefas reais, como configurar o ambiente e abrir o primeiro PR, e descreveram os bloqueios, as ferramentas abertas, as buscas realizadas e as interrupções a colegas.

Entrevistas semiestruturadas. Foram simuladas 12 entrevistas de 40 a 60 minutos, com os perfis da tabela abaixo. O roteiro abordou a experiência de entrada, as fontes de informação usadas, os momentos de maior frustração e o que o participante faria diferente.

Perfil entrevistado	Quantidade	Foco do roteiro
Desenvolvedores com até 6 meses de casa	6	Primeiras semanas, setup, primeiro PR
Tech Leads	3	Perguntas recorrentes, custo de interrupção
Engineering Managers	2	Métricas de ramp-up, previsibilidade
Onboarding Mentor	1	Rotina de acompanhamento, materiais usados

As falas foram organizadas em um diagrama de afinidade, que agrupou observações semelhantes em temas. Os temas mais recorrentes alimentaram o Mapa de Empatia e a persona.

2.2 Persona

Lucas, 27 anos, Desenvolvedor Backend Pleno. Chegou de uma startup pequena para uma empresa com mais de 80 microsserviços. Domina Java e Spring, mas nunca trabalhou com mensageria em escala. Quer mostrar resultado rápido e tem receio de parecer lento ou fazer "perguntas bobas" ao Tech Lead.

2.3 Mapa de Empatia

Quadrante	Principais achados
O que ouve	"A documentação tá no Confluence, mas metade tá desatualizada." · "Pergunta pro Fulano, ele é quem sabe desse serviço." · "Esse erro de setup é normal, todo mundo passa por isso."
O que vê	Centenas de repositórios sem diagrama de dependências claro · READMEs com comandos que não funcionam mais · Colegas experientes sempre ocupados em reuniões
O que pensa e sente	Ansiedade por não conseguir rodar o projeto localmente (setup com erros) · Medo de ser julgado improdutivo · Sensação de estar "perdido no código"
O que fala e faz	Abre dezenas de abas tentando reconstruir o fluxo de uma requisição · Copia configurações de colegas pelo Slack · Adia perguntas até ficar travado por horas
Dores	Setup de 3 a 5 dias · Informação espalhada em 5 ou mais ferramentas · Ausência de visão do todo da arquitetura
Ganhos desejados	Um ponto único de partida · Saber o próximo passo sem perguntar · Fazer o primeiro deploy na primeira semana

2.4 Principais insights

1. **A confiança na documentação é binária.** Ao encontrar a primeira página errada, o desenvolvedor deixa de confiar em todas as outras.
2. **O setup local é o primeiro "momento da verdade".** Falhar nele nos primeiros dias gera ansiedade desproporcional e marca a percepção sobre a empresa.
3. **Falta a visão do todo.** Os devs entendem o serviço em que trabalham, mas não como ele se conecta ao resto do sistema.
4. **Perguntar tem custo social.** Novos contratados preferem ficar travados a interromper um sênior repetidas vezes.
5. **Os líderes repetem as mesmas respostas.** Os Tech Leads estimam que cerca de 70% das dúvidas se repetem a cada contratação.

2.5 Pergunta norteadora (How Might We)

Como poderíamos dar ao novo desenvolvedor um caminho guiado e confiável, do setup ao primeiro commit em produção, sem depender da disponibilidade dos engenheiros sêniores?

Etapa 3 — Sessões de Ideação e Conceituação

A sessão de ideação gerou 32 ideias brutas, agrupadas em 6 temas e convergidas para um conceito único: um portal de onboarding que une setup guiado, mapa visual da arquitetura e conhecimento de domínio.

3.1 Metodologia aplicada: Diamante Duplo

O trabalho seguiu o modelo do Diamante Duplo (DESIGN COUNCIL, 2005), uma abordagem de Design Thinking (BROWN, 2010) que alterna momentos de divergência (abrir possibilidades) e convergência (fazer escolhas). O primeiro diamante trata do problema; o segundo, da solução.

Fase	Movimento	O que o grupo fez	Saída
1. Descobrir	Divergência	Investigação contextual e 12 entrevistas (Etapa 2)	Falas, observações, dados brutos
2. Definir	Convergência	Diagrama de afinidade, Mapa de Empatia, persona	Pergunta How Might We
3. Desenvolver	Divergência	Brainwriting 6-3-5, brainstorming, mapa mental	32 ideias brutas
4. Entregar	Convergência	Votação, SWOT, matriz de priorização, protótipo	Conceito DevOnboard e protótipo no Figma

3.2 Estrutura da sessão

A sessão foi realizada em formato remoto, com duração de 3 horas, usando um quadro colaborativo (Miro/FigJam). Participaram os 6 integrantes do grupo e 2 convidados: um Tech Lead e uma desenvolvedora recém-contratada, entrevistados na Etapa 2.

1. Aquecimento (15 min): revisão da persona e da pergunta How Might We.
2. Brainwriting 6-3-5 (35 min): geração silenciosa e individual de ideias.
3. Brainstorming aberto (30 min): discussão e combinação de ideias, sem crítica.
4. Mapa mental (30 min): organização das ideias em ramificações temáticas.
5. Convergência (50 min): votação por pontos, matriz de priorização e análise SWOT.
6. Fechamento (20 min): definição do conceito e dos módulos do MVP.

3.3 Brainstorming e Brainwriting

No Brainwriting 6-3-5 (ROHRBACH, 1969), cada um dos 6 participantes escreveu 3 ideias em 5 minutos e passou a folha adiante, construindo sobre as ideias dos colegas. A técnica foi escolhida porque reduz o viés de quem fala mais alto e garante voz aos participantes menos experientes, como a desenvolvedora recém-contratada. Em seguida, uma rodada de brainstorming aberto combinou e expandiu as ideias.

O foco foi a dor central: a documentação estática que envelhece. As ideias mais representativas foram:

#	Ideia	Tema
1	Checklist de onboarding interativo com progresso visível	Trilha guiada
2	Script único de setup com validação automática do ambiente	Setup
3	Diagrama de arquitetura gerado a partir dos repositórios	Arquitetura
4	Mapa clicável de microsserviços com linguagem, banco e APIs	Arquitetura
5	Glossário de domínio ligado aos serviços que usam cada termo	Regras de negócio
6	Fórum de dúvidas assíncronas com respostas reaproveitáveis	Conhecimento
7	Selo de "documentação desatualizada" a partir da data do último commit	Manutenção
8	Gamificação com marcos (primeiro build, primeiro PR, primeiro deploy)	Engajamento
9	"Good first issues" curadas por time	Primeiro commit
10	Painel para o gestor acompanhar o progresso dos novos devs	Gestão

#	Ideia	Tema
11	Chatbot com IA treinado na documentação interna	Conhecimento
12	Ambiente de desenvolvimento em nuvem pré-configurado	Setup

3.4 Convergência: votação e priorização

Cada participante recebeu 5 votos (dot voting). As ideias mais votadas foram avaliadas por impacto na dor do usuário e por esforço de implementação, em escala de 1 a 5.

Ideia	Impacto (1–5)	Esforço (1–5)	Decisão
Checklist interativo de onboarding	5	2	MVP
Mapa clicável de microsserviços	5	3	MVP
Glossário de domínio e dúvidas assíncronas	4	2	MVP
Selo de documentação desatualizada	4	2	MVP (recurso de apoio)
Gamificação com marcos	3	2	MVP (leve)
Chatbot com IA	4	5	Versão futura
Ambiente em nuvem pré-configurado	5	5	Versão futura

O chatbot e o ambiente em nuvem têm alto impacto, mas exigem infraestrutura e dados que o MVP ainda não teria. Ficaram no roadmap.

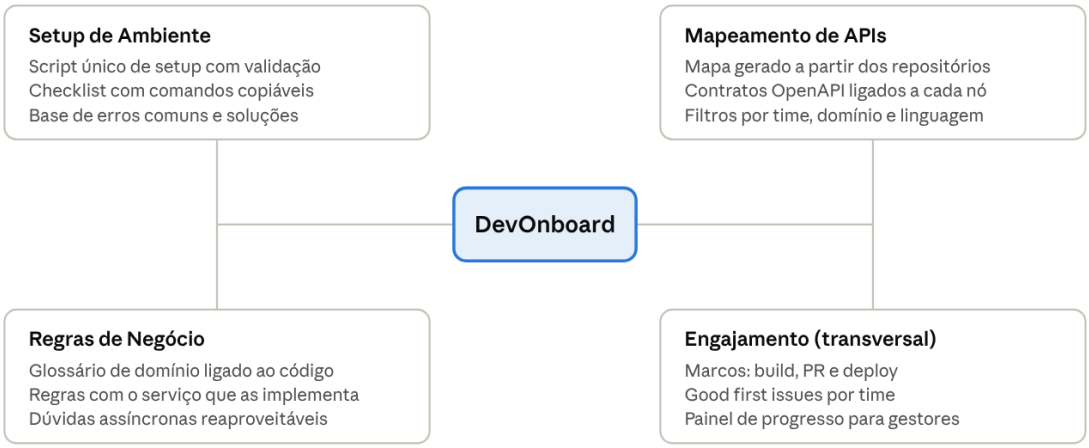
3.5 Matriz SWOT da solução proposta

	Fatores positivos	Fatores negativos
Internos	**Forças:** visualização interativa da arquitetura em tempo real · redução da dependência de sêniores · ponto único de entrada para o novo dev · métricas de onboarding mensuráveis	**Fraquezas:** necessidade de manutenção do mapeamento técnico · valor depende da qualidade do conteúdo inicial · curva de adoção para os times que alimentam o portal
Externos	**Oportunidades:** integração com repositórios GitHub/GitLab para gerar o mapa automaticamente · crescimento do trabalho remoto e híbrido · interesse do mercado em Developer Experience (DevEx) e plataformas internas	**Ameaças:** resistência cultural em manter guias de onboarding atualizados · concorrência de portais de desenvolvedor existentes (ex.: Backstage) · restrições de segurança para acessar repositórios

Leitura estratégica. As forças respondem diretamente aos insights da Etapa 2. A principal fraqueza (manutenção) e a principal ameaça (cultura) têm a mesma raiz: conteúdo que depende de esforço manual. Por isso, a integração com GitHub/GitLab deixou de ser apenas uma oportunidade e virou requisito de arquitetura: o mapa deve ser gerado e atualizado a partir do código sempre que possível.

3.6 Mapa mental de ideação

O mapa mental organizou as ideias em torno do conceito central, em três ramificações principais que correspondem aos eixos do problema (Setup de Ambiente, Mapeamento de APIs e Regras de Negócio) e uma ramificação transversal de engajamento.



Cada ramificação reúne as ideias do brainwriting ligadas a ela. As três principais viraram os três módulos do MVP, e o engajamento atravessa todos eles.

Etapa 4 — Seleção e Refinamento da Ideia

O DevOnboard foi escolhido por aliar viabilidade técnica imediata a alto valor para o time: seus três módulos do MVP usam tecnologias maduras e atacam os cinco insights da pesquisa.

4.1 Justificativa da escolha

A escolha considerou quatro critérios, avaliados de 1 a 5 contra as duas alternativas que chegaram à rodada final.

Critério	DevOnboard (portal visual)	Chatbot com IA	Ambiente em nuvem pré-configurado
Valor para o usuário	5	4	4
Viabilidade técnica imediata	5	2	2
Custo de implantação	4	2	1
Aderência aos insights da pesquisa	5	3	2
Total	**19**	**11**	**9**

O DevOnboard resolve a dor de orientação (onde começar, o que vem depois) e a de visão do todo (como o sistema se conecta). As alternativas resolvem partes do problema e dependem de pré-requisitos caros. Ambas, porém, se encaixam como evoluções futuras da plataforma: o chatbot pode consumir o conteúdo estruturado do portal, e o ambiente em nuvem pode substituir o passo de setup local.

4.2 Proposta de valor

> *"Transformar semanas de onboarding em dias, através de um portal visual e gamificado que guia o desenvolvedor do setup do ambiente até seu primeiro commit em produção."*

A proposta foi estruturada com base no Value Proposition Design (OSTERWALDER et al., 2014).

Para quem	Dor que resolve	Ganho gerado
Novo desenvolvedor	Não sabe por onde começar; setup quebra	Caminho guiado e progresso visível
Tech Lead	Responde às mesmas perguntas a cada contratação	Tempo de foco devolvido
Engineering Manager	Ramp-up imprevisível e invisível	Métricas de onboarding por pessoa e por time
Onboarding Mentor	Sem roteiro padronizado	Trilha pronta para acompanhar

4.3 Software Architecture Canvas

O Software Architecture Canvas consolidou, em uma visão única, as decisões que guiam a construção do MVP.

Bloco do canvas	Conteúdo
Propósito	Reduzir o Time-to-First-PR de novos desenvolvedores e o débito de conhecimento da organização
Stakeholders	Novos devs, Tech Leads, Engineering Managers, Onboarding Mentors, equipe de Plataforma
Requisitos funcionais	Trilha de onboarding com checklist; mapa interativo de serviços; glossário e regras de negócio; fórum de dúvidas; painel de progresso
Atributos de qualidade	Usabilidade (primeira tarefa sem treinamento); atualidade (mapa sincronizado com os repositórios); segurança (SSO e permissões por time); disponibilidade de 99,5%
Restrições	MVP em 3 meses; equipe de 4 pessoas; acesso apenas de leitura aos repositórios; dados hospedados na nuvem da empresa
Decisões arquiteturais	Aplicação web SPA + API REST; catálogo de serviços em arquivo devonboard.yaml versionado em cada repositório ("documentação como código"); sincronização por webhooks do GitHub/GitLab
Integrações	GitHub/GitLab (repositórios e webhooks); SSO corporativo (OIDC); Slack (notificações); Confluence (importação inicial de conteúdo)
Tecnologias candidatas	React + TypeScript; biblioteca de grafos (React Flow ou Cytoscape.js); Node.js (NestJS); PostgreSQL; busca com Elasticsearch/OpenSearch
Riscos	Baixa adesão dos times ao arquivo de catálogo; conteúdo inicial pobre; sobreposição com portais existentes
Métricas	Time-to-First-PR; dias de setup; chamados internos de suporte; satisfação (eNPS do onboarding)

4.4 Módulos do sistema

Módulo 1 — Hub de Onboarding (Checklist Interativo). Trilha personalizada por papel (frontend, backend, dados) e por time. Organiza o onboarding em marcos: acessos, setup local, primeiro build, primeiro PR e primeiro deploy. Cada passo traz instruções, comandos copiáveis e um critério de conclusão. Alguns passos são validados automaticamente; por exemplo, o primeiro PR é detectado via integração com o repositório.

Módulo 2 — Visualizador Dinâmico de Arquitetura (Nós e APIs). Grafo interativo em que cada nó é um serviço, banco ou fila, e cada aresta é uma dependência (chamada REST, gRPC, evento). Os dados vêm do arquivo devonboard.yaml de cada repositório, lido por webhooks. Assim, o mapa acompanha o código, atacando a fraqueza de manutenção identificada na SWOT.

Módulo 3 — Guia de Regras de Negócio e Padrões de Código. Glossário de domínio (por exemplo, o que é "pedido conciliado"), regras de negócio ligadas aos serviços que as implementam e padrões de código do time. Inclui um espaço de dúvidas assíncronas: cada resposta vira conteúdo pesquisável para as próximas contratações.

Recursos transversais. Busca global, autenticação via SSO, selo de "conteúdo possivelmente desatualizado" e painel de progresso para gestores.

4.5 Visão lógica da arquitetura



Os três módulos compartilham uma única API. A integração com GitHub/GitLab, em destaque, é a decisão que ataca a fraqueza de manutenção apontada na SWOT.

Etapa 5 — Prototipagem Não Funcional no Figma

O protótipo é não funcional: as telas usam dados fictícios e a navegação entre elas é simulada, sem back-end. Ele cobre a jornada principal do novo desenvolvedor em três telas, ligadas por uma navegação lateral fixa e pela busca global.

Etapa da engenharia de software atendida. O DevOnboard apoia a etapa de manutenção e evolução do ciclo de vida do software, em especial a compreensão do sistema existente (documentação e arquitetura). Com isso, novos membros passam a contribuir com código sem depender dos engenheiros sêniores.

5.1 Estrutura de informação

A arquitetura de informação foi organizada a partir de um card sorting aberto, simulado com 5 desenvolvedores. O resultado definiu quatro áreas de primeiro nível, sempre acessíveis pela barra lateral.

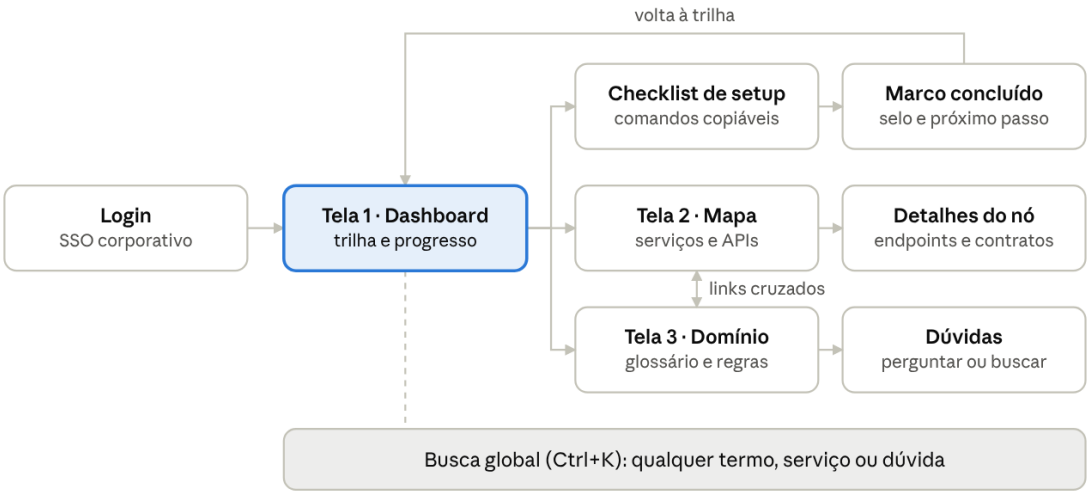
Área (menu lateral)	Conteúdo	Módulo
Minha Trilha	Dashboard, checklist de setup, marcos e próximos passos	Hub de Onboarding
Arquitetura	Mapa de serviços, detalhes de cada nó, contratos de API	Visualizador de Arquitetura
Domínio	Glossário, regras de negócio, padrões de código	Guia de Regras de Negócio
Dúvidas	Perguntas assíncronas, respostas aceitas, histórico	Guia de Regras de Negócio

A **busca global** (atalho Ctrl+K) atravessa todas as áreas. Um termo como "conciliação" retorna o verbete do glossário, os serviços que o implementam e as dúvidas já respondidas sobre ele.

5.2 Fluxo de navegação

O fluxo principal começa no login via SSO e leva ao Dashboard. De lá, o desenvolvedor segue a trilha; cada passo pode levar ao mapa de arquitetura ou à central de domínio quando ele precisa de contexto, e sempre retorna à trilha.

Toda jornada parte do Dashboard e volta à trilha



O Dashboard é o centro da jornada: mapa e domínio se ligam entre si, e cada marco concluído devolve o desenvolvedor à trilha.

5.3 Tela 1 — Dashboard do Desenvolvedor

Objetivo: responder em segundos "onde estou e qual o meu próximo passo?".

Layout e componentes:

- **Cabeçalho de boas-vindas:** saudação personalizada ("Bom dia, Lucas. Vamos colocar tudo para rodar?"), nome do mentor com botão de contato e dia do onboarding ("Dia 3 de 10").
- **Barra de progresso da trilha:** percentual concluído e linha do tempo com os cinco marcos: Acessos, Setup Local, Primeiro Build, Primeiro PR e Primeiro Deploy. Marcos concluídos recebem selo; o atual fica destacado.
- **Card "Próximo passo":** a tarefa atual em destaque, com tempo estimado e botão "Começar".
- **Checklist de setup local:** itens como instalar o Docker, clonar o repositório, configurar variáveis de ambiente e rodar a suíte de testes. Cada item tem comandos copiáveis com um clique, status (pendente, em andamento, concluído, com erro) e link "Problemas comuns" com as soluções mais frequentes.
- **Painel lateral "Seu time":** serviços sob responsabilidade do time (atalho para o mapa) e uma good first issue sugerida.

Interações prototipadas: marcar item como concluído (animação do progresso); expandir um item com erro para ver a solução; clicar em um serviço e ir ao mapa já focado nele.

5.4 Tela 2 — Mapa Interativo de Arquitetura e Microsserviços

Objetivo: dar a visão do todo e permitir entender "quem chama quem" sem ler código.

Layout e componentes:

- **Canvas central com o grafo:** nós representam serviços, bancos de dados, filas e sistemas externos, cada tipo com ícone e forma próprios. Arestas indicam o tipo de comunicação: REST (linha contínua), gRPC (tracejada) e eventos (pontilhada com seta).
- **Filtros superiores:** por time, por domínio (Pagamentos, Catálogo, Usuários), por linguagem e por tipo de comunicação. Um botão "Meu time" isola os serviços do usuário e seus vizinhos diretos.

- **Painel de detalhes do nó (ao clicar):** nome e descrição do serviço; linguagem e framework (ex.: Java 21 / Spring Boot); banco associado (ex.: PostgreSQL); filas consumidas e produzidas; lista de endpoints com link para o contrato OpenAPI; time responsável e canal de Slack; links para repositório, dashboard de monitoramento e runbook.
- **Selo de atualização:** "Sincronizado com o repositório há 2 horas", reforçando a confiança no conteúdo.
- **Minimapa e controles de zoom** para navegar em arquiteturas grandes.

Interações prototipadas: clicar em um nó destaca suas dependências e esmaece o restante; clicar em um endpoint abre seu contrato; o modo "Seguir uma requisição" anima o caminho de um fluxo de negócio (ex.: "Finalizar compra") pelos serviços envolvidos.

5.5 Tela 3 — Central de Regras de Negócio e Glossário de Domínio

Objetivo: traduzir a linguagem do negócio para o código e reduzir perguntas repetidas.

Layout e componentes:

- **Busca e índice alfabético** do glossário, com filtro por domínio.
- **Verbete do termo:** definição em linguagem simples, exemplo prático, regras de negócio associadas (ex.: "Um pedido só é conciliado após a confirmação do adquirente"), serviços que implementam a regra (com link para o mapa) e trecho de código de referência.
- **Padrões de código do time:** convenções de nomenclatura, estrutura de pastas, padrão de commits e checklist de code review.
- **Ambiente de dúvidas assíncronas:** campo "Faça sua pergunta" com sugestão automática de dúvidas parecidas antes de publicar; lista de perguntas com status (aberta, respondida, resposta aceita); votos de utilidade; opção de perguntar de forma anônima, reduzindo o custo social identificado na pesquisa.
- **Selo de revisão:** data da última revisão do verbete e responsável. Verbetes sem revisão há mais de 90 dias recebem o selo "possivelmente desatualizado".

Interações prototipadas: buscar um termo; abrir um verbete e navegar até o serviço no mapa; publicar uma dúvida e ver o estado "respondida".

5.6 Decisões de design de interface

- **Sistema visual:** paleta neutra com uma cor de destaque para ações e progresso; cores de status (verde, amarelo, vermelho) usadas sempre com ícone e texto, nunca sozinhas, por acessibilidade.
- **Tipografia:** fonte sem serifa para interface e fonte monoespaçada para comandos e código.
- **Componentes reutilizáveis no Figma:** cards, itens de checklist com variantes de estado, nós do grafo, selos e painel lateral, organizados com Auto Layout e Variants.
- **Modo escuro:** previsto, por ser preferência comum entre desenvolvedores.
- **Gamificação leve:** selos por marco, ofensiva pessoal de dias em sequência e mensagem de celebração no primeiro deploy, sem rankings entre pessoas, para não aumentar a ansiedade identificada na empatia.

5.7 Validação do protótipo

A validação foi simulada em um teste de usabilidade com 5 desenvolvedores, em sessões moderadas de 20 minutos e com três tarefas: encontrar o próximo passo do setup; descobrir qual banco o serviço de pagamentos usa; e achar o significado de "pedido conciliado". No cenário simulado, todos concluíram as tarefas 1 e 3, e 4 de 5 concluíram a tarefa 2. O principal ajuste foi tornar o botão "Meu time" mais visível no mapa, pois a quantidade de nós intimidou na primeira visualização. Esses resultados são ilustrativos e devem ser confirmados em testes reais no piloto da Etapa 6.

5.8 Link do protótipo

Protótipo no Figma: https://high-mind-58758181.figma.site/

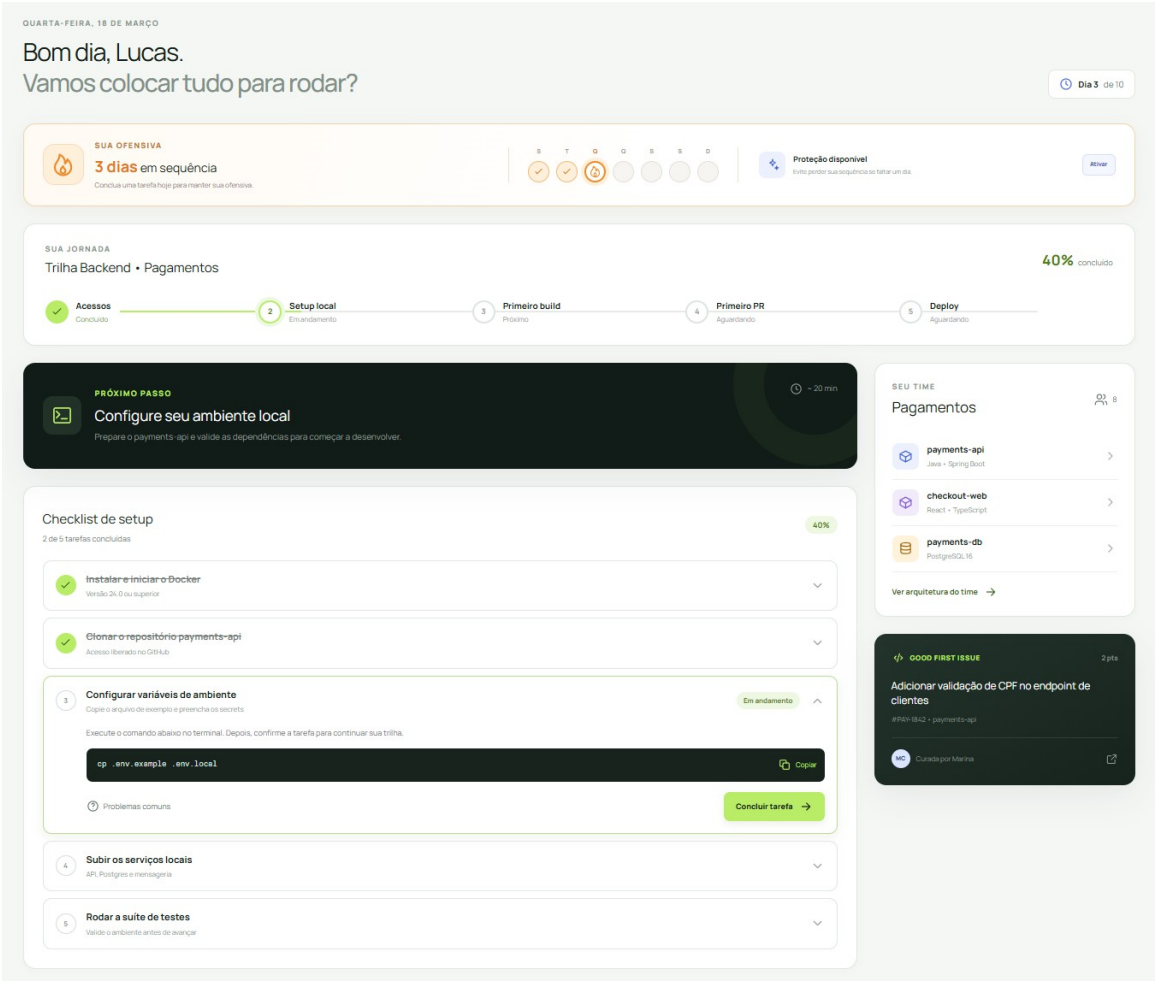


Figura 4 — Tela 1: Dashboard do desenvolvedor.

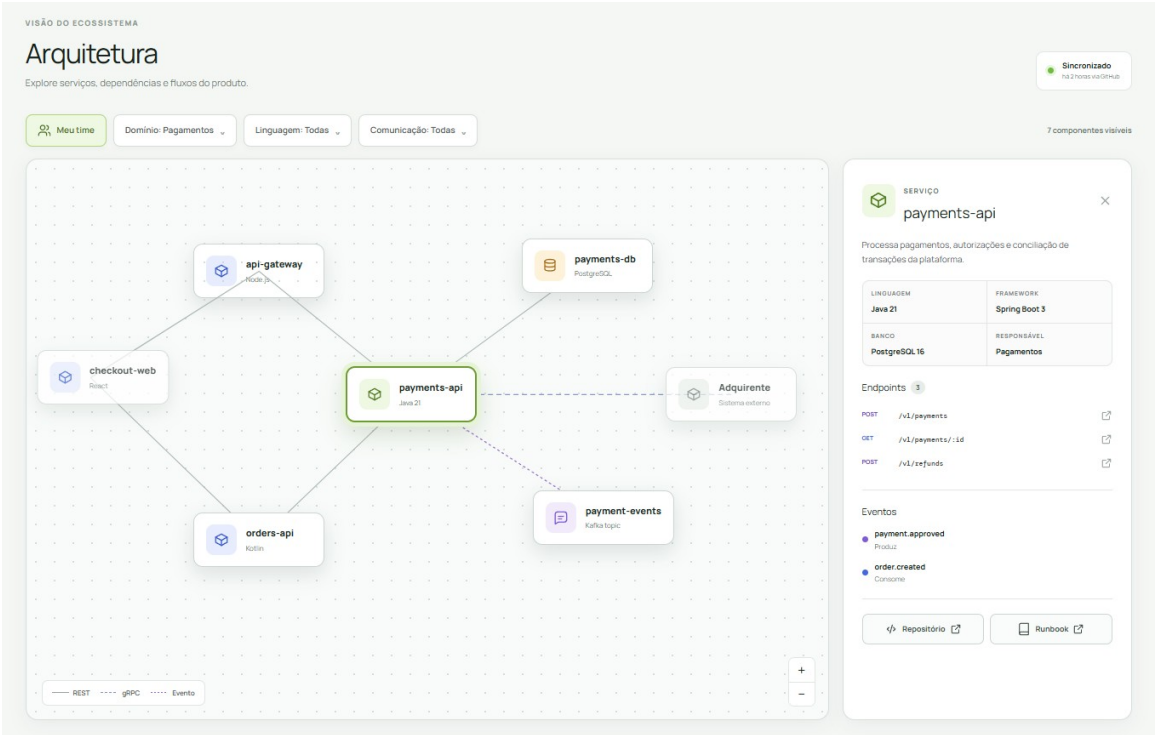


Figura 5 — Tela 2: Mapa interativo de arquitetura.

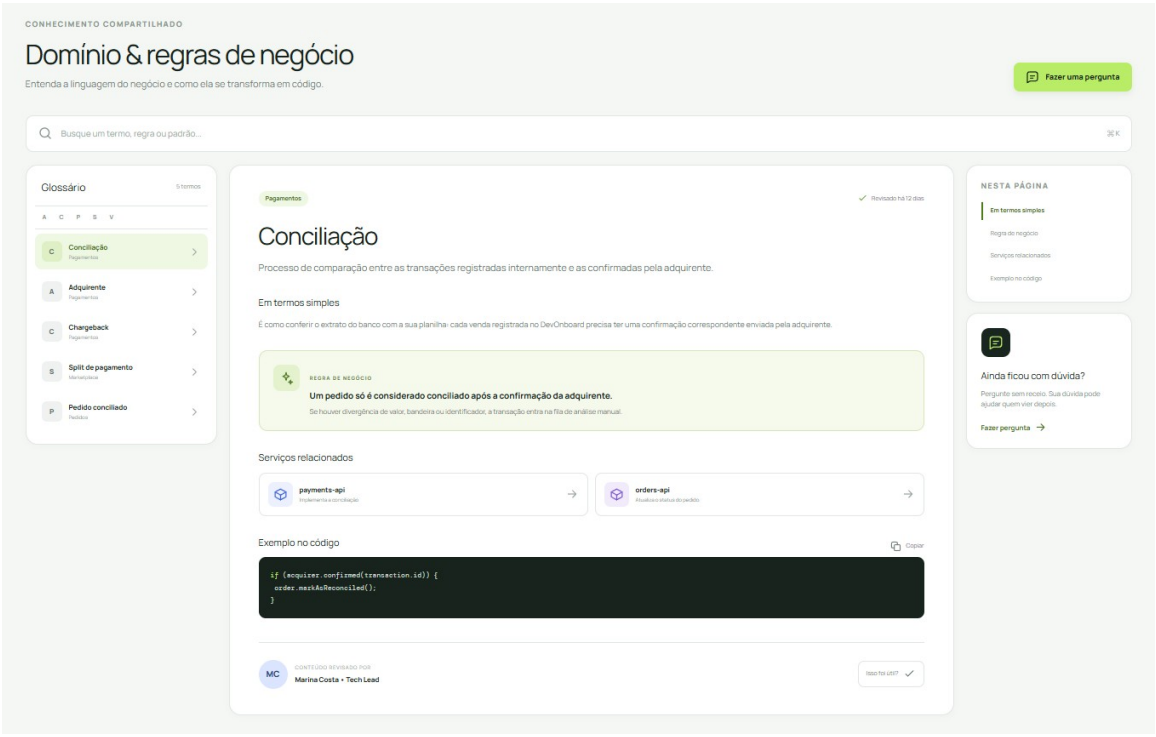


Figura 6 — Tela 3: Central de regras de negócio e glossário de domínio.

Etapa 6 — Conclusão, Métricas de Sucesso e Lições Aprendidas

O DevOnboard projeta reduzir o Time-to-First-PR em 60% (de 4,5 semanas para menos de 2) e os chamados internos de suporte técnico em 45%, devolvendo tempo de foco aos sêniores.

6.1 Métricas esperadas de impacto

Métrica	Linha de base (simulada)	Meta após 6 meses	Variação	Como medir
Time-to-First-PR	22 dias úteis	9 dias úteis	−60%	Data de admissão vs. primeiro PR aprovado (API do GitHub/GitLab)
Chamados internos de suporte técnico por novo dev	40 no 1º mês	22 no 1º mês	−45%	Perguntas em canais de ajuda + tickets internos
Tempo de setup do ambiente local	4 dias	1 dia	−75%	Conclusão do marco "Setup Local" no Hub
Horas de sênior por contratação (1º mês)	30 h	15 h	−50%	Registro do mentor + pesquisa com Tech Leads
Satisfação com o onboarding (eNPS)	+10	+45	+35 pontos	Pesquisa ao fim do 30º dia

Retorno estimado. No cenário de referência da Etapa 1 (30 contratações por ano e custo oculto de cerca de R$ 20.000 por contratação), uma redução de 60% no tempo improdutivo representa uma economia potencial na ordem de R$ 300.000 a R$ 360.000 por ano, sem contar o efeito sobre a retenção.

Plano de medição. As métricas serão coletadas em um piloto com dois times durante um trimestre, comparando novas contratações com o histórico dos 12 meses anteriores. As metas acima são hipóteses a serem validadas nesse piloto.

6.2 Próximos passos (roadmap)

1. **MVP (meses 1–3):** Hub de Onboarding, mapa a partir do devonboard.yaml e glossário com dúvidas assíncronas.

2. **Piloto (meses 4–6):** dois times, coleta das métricas e ajustes de usabilidade.
3. **Evolução (meses 7–12):** assistente com IA alimentado pelo conteúdo do portal e integração com ambientes de desenvolvimento em nuvem.

6.3 Lições aprendidas

Ouvir os desenvolvedores evitou o escopo inflado. A primeira versão da ideia, antes da fase de empatia, incluía chatbot com IA, ambiente em nuvem, gamificação com ranking e integração com dez ferramentas. A pesquisa mostrou que a dor principal era mais simples: saber o próximo passo e confiar no que se lê. Isso permitiu cortar o overengineering e aplicar o conceito de Mínimo Produto Viável (RIES, 2012): três módulos que entregam valor desde a primeira semana.

Divergir antes de convergir gera ideias melhores. O Brainwriting 6-3-5 deu voz à desenvolvedora recém-contratada, autora da ideia do selo de "documentação desatualizada", hoje uma das funcionalidades mais valorizadas na validação do protótipo.

A SWOT mudou a arquitetura. Ao cruzar a fraqueza de manutenção com a ameaça cultural, o grupo transformou a integração com repositórios em requisito central. A ferramenta de análise estratégica influenciou diretamente uma decisão técnica.

Prototipar barato e cedo reduz risco. A validação revelou que o mapa completo intimidava na primeira visualização. Corrigir isso no Figma custou minutos; no código, custaria uma sprint.

Gamificação exige cuidado. Rankings entre pessoas foram descartados porque reforçariam a ansiedade observada na empatia. A gamificação ficou restrita a marcos pessoais.

6.4 Conclusão

O onboarding técnico é um problema caro e pouco visível: dilui-se em semanas de baixa produtividade, horas de sênior e frustração silenciosa. A aplicação do Design Thinking pelo Diamante Duplo levou o grupo de uma dor difusa a uma solução focada, validada em cenário simulado com usuários e tecnicamente viável. O DevOnboard propõe tratar o conhecimento técnico como um ativo vivo, ligado ao código e organizado para quem chega. Com isso, a empresa transforma semanas de onboarding em dias e reduz o débito de conhecimento a cada nova contratação.

Referências

- BROWN, T. Design Thinking: uma metodologia poderosa para decretar o fim das velhas ideias. Rio de Janeiro: Elsevier, 2010.
- DESIGN COUNCIL. Framework for Innovation: the Double Diamond. Londres: Design Council, 2005 (rev. 2019).
- HOLTZBLATT, K.; BEYER, H. Contextual Design: Design for Life. 2. ed. Cambridge: Morgan Kaufmann, 2017.
- OSTERWALDER, A. et al. Value Proposition Design. Hoboken: Wiley, 2014.
- RIES, E. A Startup Enxuta. São Paulo: Leya, 2012.
- ROHRBACH, B. Kreativ nach Regeln: Methode 635, eine neue Technik zum Lösen von Problemen. Absatzwirtschaft, v. 12, 1969.